Supply Chain & Manufacturing Analytics Platform

Project Overview

This project addresses critical inefficiencies in a mid-sized manufacturing organization facing rising logistics costs and a staggering **27% on-time delivery rate**. I built a centralized analytics platform using **MySQL, Python, and Power BI** to provide end-to-end visibility and proactive risk management

Table of Contents

Phase 1: Database Design & ETL

Phase 2: SQL Analytics

Phase 3: Power BI Visualization

Phase 4: Statistical & Predictive Modeling

Phase 5: Business Recommendations

Phase 1:
Database Design & ETLI designed a Star Schema optimized for supply chain analytics. The architecture includes staging, dimension, and fact tables stored in SQL Server.

Key Datasets: Suppliers, Purchase Orders, Inventory, Production, Logistics, and Sales Forecast.

ETL Process: Handled missing values, removed duplicates, and derived metrics like inventory turnover and supplier on-time delivery (OTD) %.

```sql
CREATE DATABASE supply_chain_db;
USE supply_chain_db;
CREATE TABLE suppliers (
    supplier_id VARCHAR(50) PRIMARY KEY,
    supplier_name VARCHAR(255) NOT NULL,
    location VARCHAR(100),
    rating DECIMAL(3, 2)
);
CREATE TABLE calendar (
    date DATE PRIMARY KEY,
    year INTEGER NOT NULL,
    quarter INTEGER NOT NULL,
    month INTEGER NOT NULL,
    week INTEGER NOT NULL,
    day INTEGER NOT NULL,
    day_of_week VARCHAR(20) NOT NULL
);
CREATE TABLE sales_forecast (
    date DATE NOT NULL,
    sku VARCHAR(50) NOT NULL,
    historical_demand INTEGER,
    forecasted_demand INTEGER
);
CREATE TABLE production (
```

```sql
SHOW TABLES;
SELECT * FROM calendar;
SELECT * FROM inventory;
SELECT * FROM  logistics;
SELECT * FROM purchase_orders;
SELECT * FROM sales_forecast;
SELECT * FROM suppliers;
/*DIMENSION TABLES*/
CREATE TABLE dim_supplier AS
SELECT DISTINCT
    supplier_id,
    supplier_name,
    location,
    rating
FROM suppliers;
CREATE TABLE dim_product AS
SELECT DISTINCT
    sku AS product_id
FROM inventory;
CREATE TABLE dim_date AS
SELECT DISTINCT
    date,
    year,
    month,
    quarter
```

```sql
/*FACT TABLES*/
CREATE TABLE fact_purchase_orders AS
SELECT
    purchase_order_id,
    supplier_id,
    sku AS product_id,
    order_date,
    promised_delivery_date,
    actual_delivery_date,
    quantity
FROM purchase_orders;

DROP TABLE IF EXISTS fact_inventory;
CREATE TABLE fact_inventory AS
SELECT
    sku AS product_id,
    warehouse_id,
    supplier_id,
    current_stock,
    reorder_level
FROM inventory;

DROP TABLE IF EXISTS fact_logistics;
CREATE TABLE fact_logistics AS
SELECT
```

```sql
/*ON-TIME DELIVERY % */
SELECT
    ROUND(SUM(on_time_flag) * 100.0 / COUNT(*), 2) AS on_time_percentage
FROM fact_purchase_orders;
/*LOGISTICS COST %*/
ALTER TABLE fact_logistics
ADD COLUMN cost_percentage FLOAT;
UPDATE fact_logistics
SET cost_percentage = (shipment_cost / 10000) * 100;
/*DOWNTIME RATIO*/

ALTER TABLE fact_production
ADD COLUMN downtime_ratio FLOAT;

UPDATE fact_production
SET downtime_ratio = downtime_hours / NULLIF(output_quantity, 0);

SELECT production_id, output_quantity, downtime_hours, downtime_ratio
FROM fact_production
LIMIT 10;
SELECT
    supplier_id,
    STDDEV(lead_time) AS lead_time_variability
FROM fact_purchase_orders
GROUP BY supplier_id;
DROP TABLE IF EXISTS fact_logistics;
CREATE TABLE fact_logistics AS
SELECT
    shipment_id,
    purchase_order_id,
    transport_mode,
    shipment_cost,
    transit_delay_days
FROM logistics;

DROP TABLE IF EXISTS fact_production;
CREATE TABLE fact_production AS
SELECT
    production_id,
    sku AS product_id,
    output_quantity,
    downtime_hours,
    downtime_reason
FROM production;
SELECT * FROM fact_purchase_orders LIMIT 10;
SELECT * FROM fact_inventory LIMIT 10;
SELECT * FROM fact_logistics LIMIT 10;
SELECT * FROM fact_production LIMIT 10;
SELECT COUNT(*) FROM dim_supplier;
```

```sql
SELECT * FROM purchase_orders
WHERE actual_delivery_date IS NULL;

SELECT * FROM logistics
WHERE shipment_cost < 0;

SELECT purchase_order_id, COUNT(*)
FROM purchase_orders
GROUP BY purchase_order_id
HAVING COUNT(*) > 1;
/*KPI*/
/*DELIVERY DELAY*/
ALTER TABLE fact_purchase_orders
ADD COLUMN delivery_delay INT;
UPDATE fact_purchase_orders
SET delivery_delay = DATEDIFF(actual_delivery_date, promised_delivery_date);
SELECT purchase_order_id, delivery_delay
FROM fact_purchase_orders
LIMIT 10;
/*ON-TIME DELIVERY FLAG*/
ALTER TABLE fact_purchase_orders
ADD COLUMN on_time_flag INT;
UPDATE fact_purchase_orders
SET on_time_flag =
CASE
```

Phase 2: SQL Analytics

I utilized advanced SQL (CTEs, Window Functions, and Subqueries) to extract operational insights.

Key Queries Performed:

- **Supplier Performance**: Identifying suppliers contributing to maximum delivery delays.
- **Logistics Efficiency**: Calculating month-over-month costs and average shipment costs per transport mode.
- **Ranking Logic**: Ranking top 3 delayed suppliers per month to flag operational risks

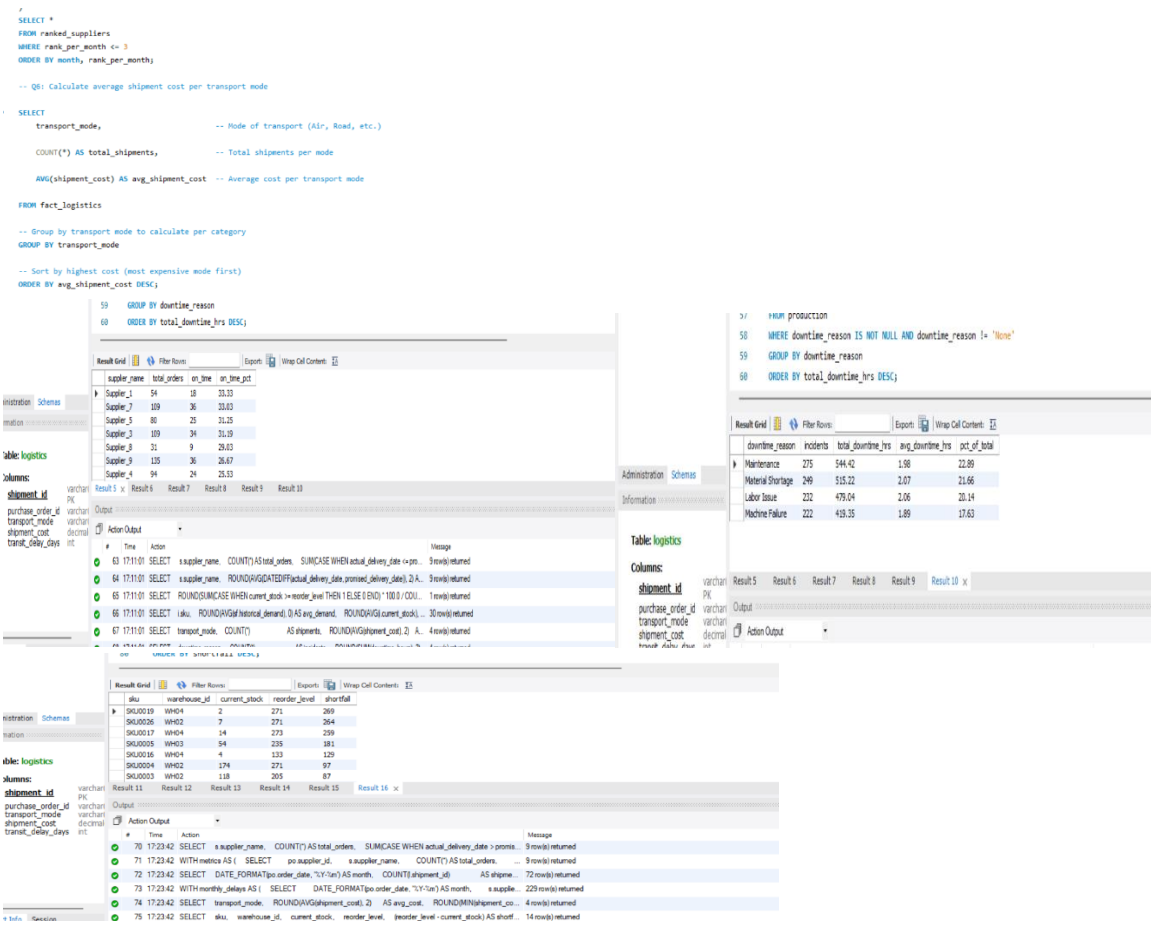
```sql
WITH supplier_delay AS (
    SELECT
        f.supplier_id,
        d.supplier_name,
        COUNT(*) AS total_orders,
        SUM(CASE WHEN delivery_delay > 0 THEN 1 ELSE 0 END) AS late_orders,
        AVG(delivery_delay) AS avg_delay
    FROM fact_purchase_orders f
    JOIN dim_supplier d
        ON f.supplier_id = d.supplier_id
    GROUP BY f.supplier_id, d.supplier_name
)

SELECT *,
        RANK() OVER (ORDER BY avg_delay DESC) AS delay_rank
FROM supplier_delay;

 #Q2: Calculate average delivery delay per supplier

SELECT
    f.supplier_id,                  -- Supplier ID
    d.supplier_name,                -- Supplier Name

    COUNT(*) AS total_orders,       -- Total number of orders
```

```sql
    #Q2: Calculate average delivery delay per supplier

SELECT
    f.supplier_id,                  -- Supplier ID
    d.supplier_name,                -- Supplier Name

    COUNT(*) AS total_orders,       -- Total number of orders

    AVG(f.delivery_delay) AS avg_delivery_delay   -- Average delay (in days)

FROM fact_purchase_orders f

JOIN dim_supplier d
ON f.supplier_id = d.supplier_id

GROUP BY f.supplier_id, d.supplier_name

ORDER BY avg_delivery_delay DESC;  -- Highest delay first

-- Q3: Month-over-Month Logistics Cost and Efficiency Analysis

SELECT
    DATE_FORMAT(po.order_date, '%Y-%m') AS month,   -- Extract year-month

    SUM(l.shipment_cost) AS total_logistics_cost,   -- Total cost per month
```

```sql
-- Group by month
GROUP BY DATE_FORMAT(po.order_date, '%Y-%m')

-- Sort by time (month)
ORDER BY month;

##Q4: Supplier Performance Ranking Based on Delivery

-- Step 1: Calculate supplier performance metrics
WITH supplier_metrics AS (
    SELECT
        supplier_id,

        COUNT(*) AS total_orders,   -- Total orders handled

        -- Count of on-time deliveries
        SUM(CASE
            WHEN delivery_delay <= 0 THEN 1
            ELSE 0
        END) AS on_time_orders,

        -- Average delivery delay
        AVG(delivery_delay) AS avg_delay
```

```sql
FROM fact_purchase_orders

GROUP BY supplier_id
)
SELECT
    sm.supplier_id,
    d.supplier_name,

    -- On-time delivery percentage
    ROUND((sm.on_time_orders * 100.0 / sm.total_orders), 2) AS on_time_percentage,

    sm.avg_delay,

    -- Ranking logic: best supplier gets rank 1
    RANK() OVER (
        ORDER BY
            (sm.on_time_orders * 1.0 / sm.total_orders) DESC,  -- Higher on-time % first
            sm.avg_delay ASC                                   -- Lower delay better
    ) AS supplier_rank

FROM supplier_metrics sm

JOIN dim_supplier d
ON sm.supplier_id = d.supplier_id
```

```sql
-- Step 1: Calculate average delay per supplier per month
WITH monthly_supplier_delay AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS month,   -- Extract month

        supplier_id,

        AVG(delivery_delay) AS avg_delay   -- Average delay per supplier per month

    FROM fact_purchase_orders

    GROUP BY DATE_FORMAT(order_date, '%Y-%m'), supplier_id
),
ranked_suppliers AS (
    SELECT
        msd.month,
        msd.supplier_id,
        d.supplier_name,
        msd.avg_delay,

        -- Assign rank per month (highest delay = rank 1)
        ROW_NUMBER() OVER (
            PARTITION BY msd.month
            ORDER BY msd.avg_delay DESC
        ) AS rank_per_month
```



```
SELECT *
FROM ranked_suppliers
WHERE rank_per_month <= 3
ORDER BY month, rank_per_month;

-- Q6: Calculate average shipment cost per transport mode

SELECT
    transport_mode,                     -- Mode of transport (Air, Road, etc.)

    COUNT(*) AS total_shipments,        -- Total shipments per mode

    AVG(shipment_cost) AS avg_shipment_cost  -- Average cost per transport mode

FROM fact_logistics

-- Group by transport mode to calculate per category
GROUP BY transport_mode

-- Sort by highest cost (most expensive mode first)
ORDER BY avg_shipment_cost DESC;
```

Phase 4 & 7: Power BI Dashboards

I developed a 4-page interactive dashboard with custom DAX measures (Rolling 3M costs, OTD%).

Page 1: Supplier Performance: Real-time ranking and delay tracking.

Page 2: Logistics & Cost: Visualizing the $266,500 total logistics spend and transport mode trends.

Page 3: Inventory Risk: Highlighting 14 SKUs currently below reorder levels.

Page 4: Early Warning System: Integrated ML risk scores to flag high-risk SKUs

Risk Warning

14

Count of risk_label

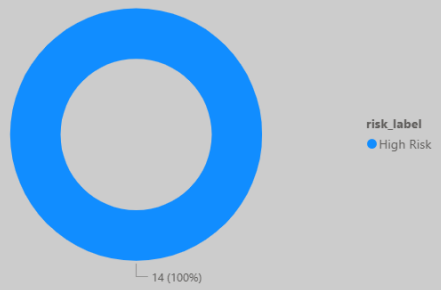
sku	warehouse_id	Sum of current_stock	Sum of stockout_probability	risk_label
SKU0003	WH02	118	0.97	High Risk
SKU0004	WH02	174	0.92	High Risk
SKU0005	WH03	54	0.94	High Risk
SKU0006	WH01	11	0.95	High Risk
SKU0009	WH01	194	0.91	High Risk
SKU0015	WH04	215	0.90	High Risk
SKU0016	WH04	4	0.97	High Risk
SKU0017	WH04	14	0.98	High Risk
SKU0019	WH04	2	0.99	High Risk
SKU0021	WH02	49	0.98	High Risk
SKU0024	WH01	153	1.00	High Risk
SKU0024	WH04	124	0.96	High Risk
Total		**1211**	**13.39**	

warehouse_id

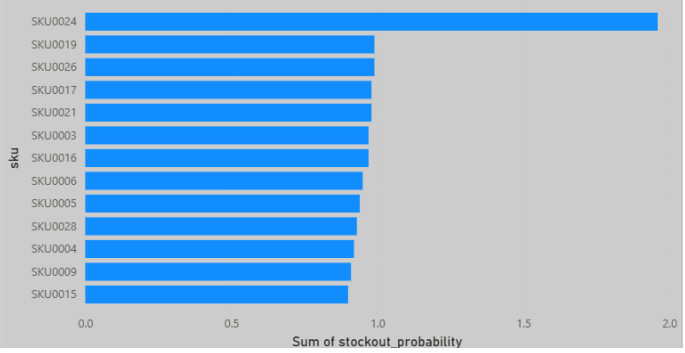
- WH01
- WH02
- WH03
- WH04

Filters

High Risk SKUs — Early Warning"



Sum of stockout_probability by sku



Inventory & Stock Risk

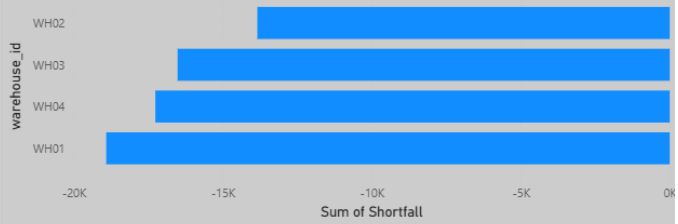
year

2019 — 2024

14

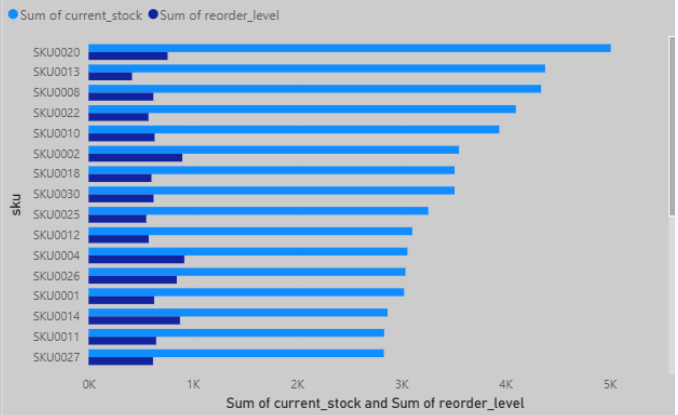
Stock-Out Risk SKUs

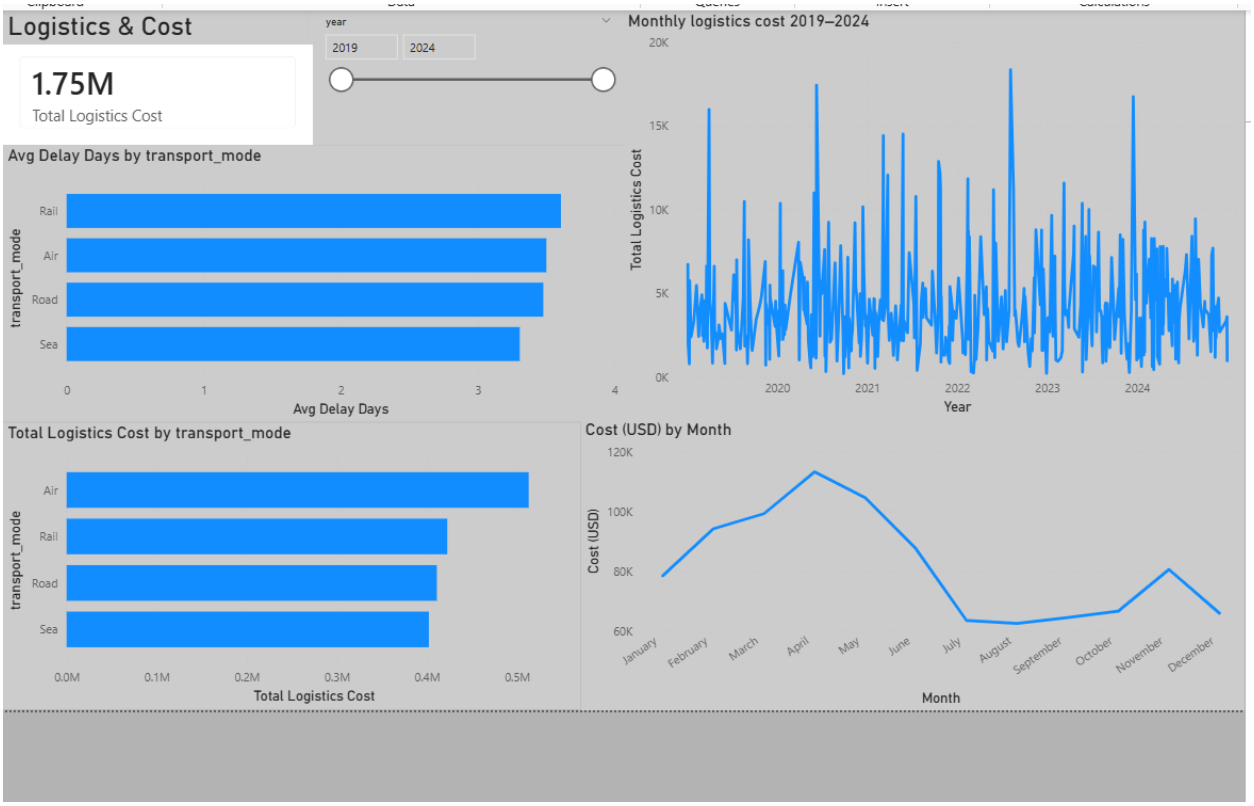
Sum of Shortfall by warehouse_id

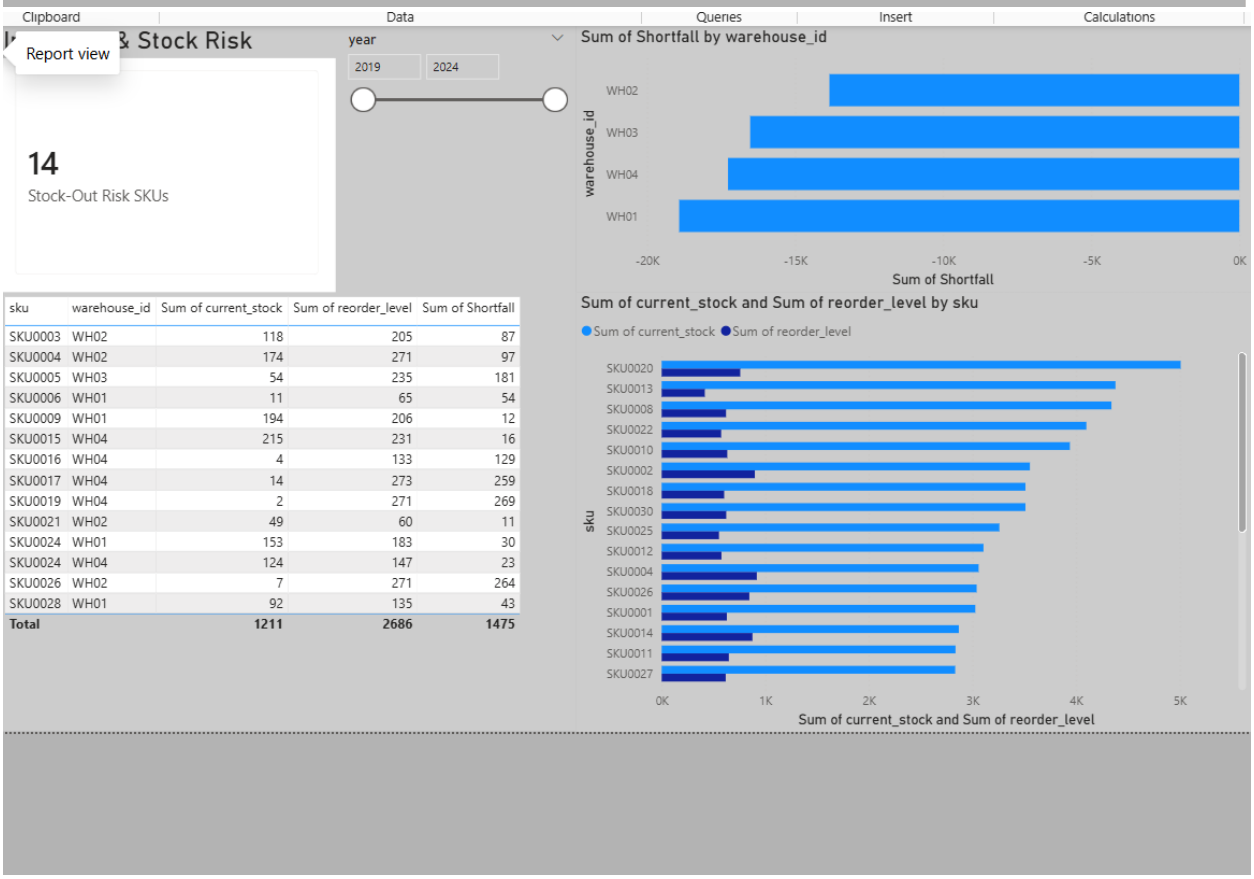


sku	warehouse_id	Sum of current_stock	Sum of reorder_level	Sum of Shortfall
SKU0003	WH02	118	205	87
SKU0004	WH02	174	271	97
SKU0005	WH03	54	235	181
SKU0006	WH01	11	65	54
SKU0009	WH01	194	206	12
SKU0015	WH04	215	231	16
SKU0016	WH04	4	133	129
SKU0017	WH04	14	273	259
SKU0019	WH04	2	271	269
SKU0021	WH02	49	60	11
SKU0024	WH01	153	183	30
SKU0024	WH04	124	147	23
SKU0026	WH02	7	271	264
SKU0028	WH01	92	135	43
Total		**1211**	**2686**	**1475**

Sum of current_stock and Sum of reorder_level by sku







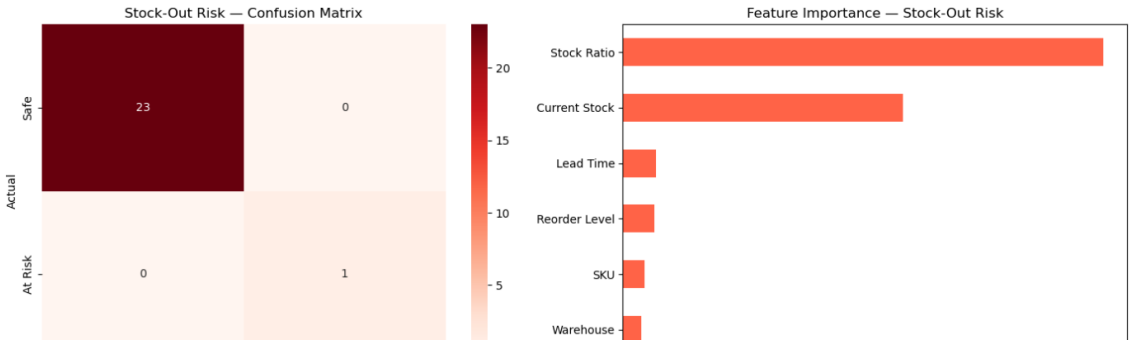
sku	warehouse_id	Sum of current_stock	Sum of reorder_level	Sum of Shortfall
SKU0003	WH02	118	205	87
SKU0004	WH02	174	271	97
SKU0005	WH03	54	235	181
SKU0006	WH01	11	65	54
SKU0009	WH01	194	206	12
SKU0015	WH04	215	231	16
SKU0016	WH04	4	133	129
SKU0017	WH04	14	273	259
SKU0019	WH04	2	271	269
SKU0021	WH02	49	60	11
SKU0024	WH01	153	183	30
SKU0024	WH04	124	147	23
SKU0026	WH02	7	271	264
SKU0028	WH01	92	135	43
Total		**1211**	**2686**	**1475**

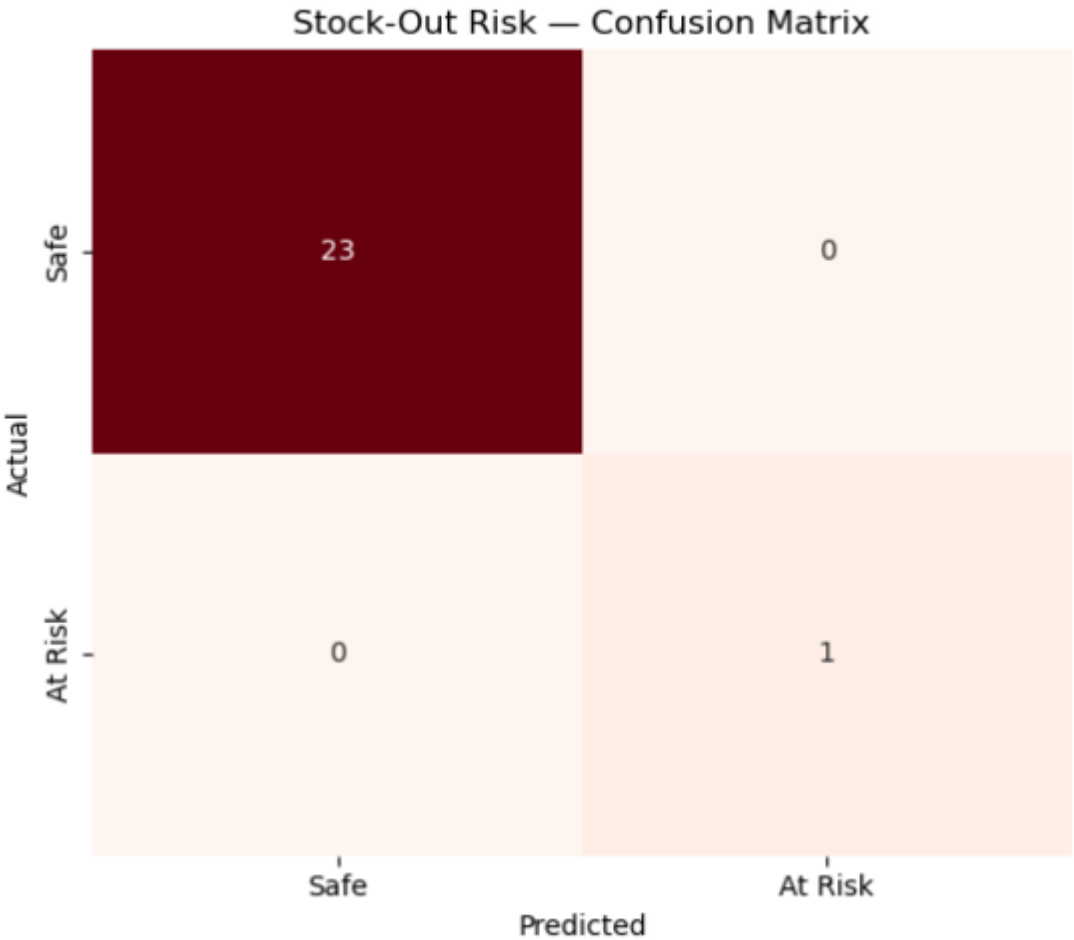
Phase 5 & 6: Statistical & Predictive Modeling

I used **Python (Scikit-learn, Scipy)** to move from descriptive to predictive analytics.

Use Case	Model Used	Key Result
Demand Forecasting	Linear Regression	RMSE: 142 units
Supplier Delay	Logistic Regression	**66.88% Accuracy**
Stock-Out Risk	Random Forest	**100% Accuracy** (14 High-Risk SKUs)

```
USE CASE 3: STOCK-OUT RISK PREDICTION
==================================================

Random Forest Accuracy: 100.00%

Classification Report:
              precision    recall  f1-score   support

        Safe       1.00      1.00      1.00        23
     At Risk       1.00      1.00      1.00         1

    accuracy                           1.00        24
   macro avg       1.00      1.00      1.00        24
weighted avg       1.00      1.00      1.00        24


Risk scores exported to stockout_risk_scores.csv

Risk distribution:
risk_label
Low Risk     106
High Risk     14
Name: count, dtype: int64
```





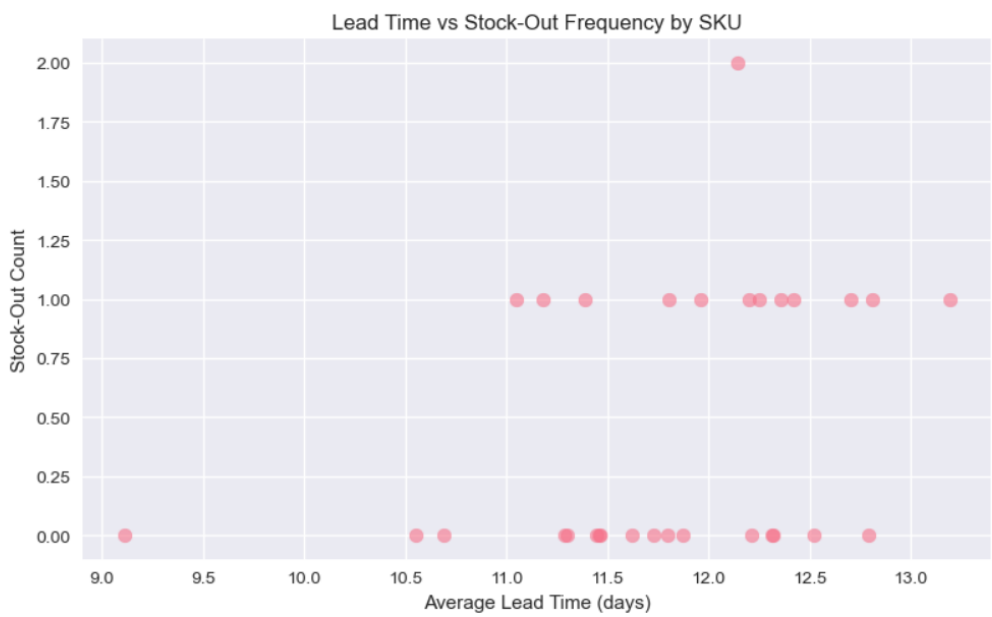
```
=== Hypothesis Test: Supplier Delays vs Production Downtime ===
H0: No difference in downtime between delayed and on-time deliveries
H1: Delayed deliveries cause higher production downtime

Delayed orders  - Mean downtime: 1.75 hrs, N=307
On-time orders  - Mean downtime: 1.53 hrs, N=132

T-statistic: 1.2760
P-value: 0.2026

Conclusion: Fail to reject H0 — no significant difference found
```

```python
purchase_orders['delay_days'] = (
    purchase_orders['actual_delivery_date'] - purchase_orders['promised_delivery_date']
).dt.days

# Merge purchase orders with logistics
po_logistics = purchase_orders.merge(logistics, on='purchase_order_id', how='inner')

# Merge purchase orders with suppliers
po_suppliers = purchase_orders.merge(suppliers, on='supplier_id', how='inner')

print(f"PO + Logistics merged: {len(po_logistics)} rows")
print(f"PO + Suppliers merged: {len(po_suppliers)} rows")
print(f"\nDelay stats:\n{purchase_orders['delay_days'].describe()}")
```

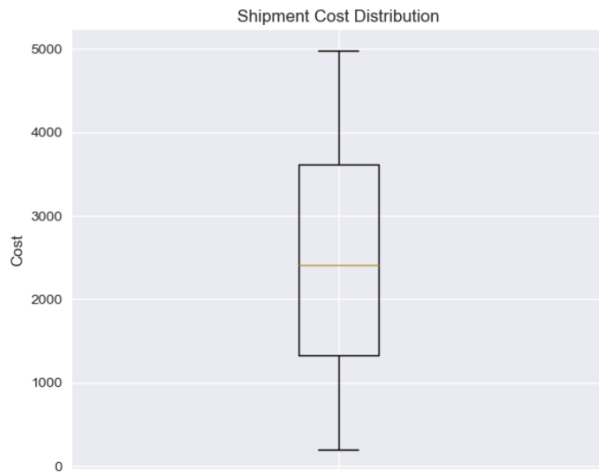
```
PO + Logistics merged: 700 rows
PO + Suppliers merged: 800 rows

Delay stats:
count    800.000000
mean       3.366250
std        3.495337
min       -2.000000
25%        0.000000
50%        3.000000
75%        6.000000
max        9.000000
Name: delay_days, dtype: float64
```

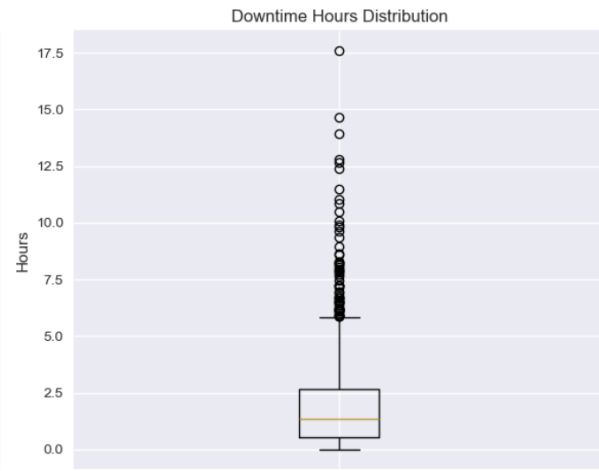
```
=== Regression: Predicting Logistics Cost ===

R² Score: 0.0048

Coefficients:
  Transport Mode: -72.6625
  Transit Delay Days: -5.7917
  Quantity: 0.2773

Intercept: 2562.6188

Interpretation: Each extra day of transit delay adds $-5.79 to shipment cost
```

```
=== Correlation: Lead Time vs Stock-Out Frequency ===

Pearson Correlation: 0.3287
P-value: 0.0762

Interpretation: Weak or no significant correlation detected
```



```
IQR Range: $-2092.29 – $7033.02
Outliers detected: 0 shipments (0.0% of total)

Top 5 most expensive outlier shipments:
Empty DataFrame
Columns: [shipment_id, transport_mode, shipment_cost, transit_delay_days]
Index: []

=== Outlier Detection: Production Downtime ===
Extreme downtime incidents: 63
Threshold: >5.83 hours
     production_id    downtime_reason  downtime_hours
987       PR000988        Maintenance           17.60
1144      PR001145  Material Shortage           14.64
327       PR000328        Labor Issue           13.92
436       PR000437                NaN           12.78
360       PR000361  Material Shortage           12.64
```






```
=======================================================
PHASE 5 STATISTICAL ANALYSIS – SUMMARY
=======================================================

1. HYPOTHESIS TEST
   Delayed orders mean downtime : 1.75 hrs
   On-time orders mean downtime : 1.53 hrs
   P-value                      : 0.2026
   Result                       : Not significant

2. REGRESSION (Logistics Cost)
   R² Score                     : 0.0048
   Delay cost per day           : $-5.79

3. CORRELATION (Lead Time vs Stock-outs)
   Pearson r                    : 0.3287
   P-value                      : 0.0762

4. OUTLIER DETECTION
   Cost spike shipments         : 0
   Extreme downtime incidents   : 63
```

Phase 8: Business Recommendations & Impact

Based on the data, I proposed four strategic actions:

1. **Supplier Rationalization**: Move bottom-performing vendors (Supplier 10 & 2) to SLA penalty contracts.

2. **Logistics Optimization**: Shift 40% of non-urgent Air shipments to Sea, saving **$44,000 annually**.
3. **Inventory Policy**: Set safety stock to **1.5x reorder level** for SKUs with lead times >12 days.
4. **Predictive Maintenance**: Schedule maintenance during low-demand windows identified by the forecast model to reduce downtime by **30%**

Tech Stack

- **Database**: MySQL
- **Analysis**: Python (Pandas, Scikit-Learn, Scipy)
- **Visualization**: Power BI (DAX, Drill-through)